2.2 UI Réaliser

IHM 1: Supervision station meteo exterieur

Présentation Générale

Cette interface de visualisation des données permet de surveiller en temps réel l'état climatique extérieur, la qualité de l'air et l'activité sismique.

Spécifications Techniques

- **Source des données :** Station Oregon + SGBD Général.
- **Protocole de communication :** MQTT (Message Queuing Telemetry Transport).
- **État de la connexion :** Monitoring en temps réel.

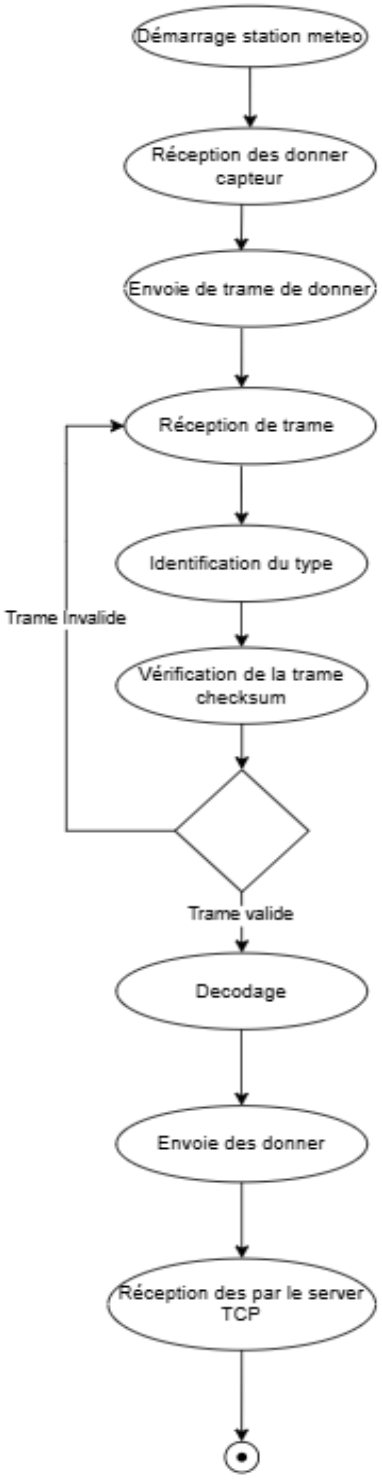
Analyse des Modules

	Données affichées	Remarques techniques
Météo Externe	Vent (km/h + Direction), Pluie (mm), Température (°C), Humidité (%)	Les données provenant du RTOS (stockées en INT) sont ici formatées avec leur précision décimale d'origine.
Qualité de l'Air	Indice UV, Particules Fines (PM 2.5)	Utilisation d'alertes textuelles et colorées (**Élevé/Mauvais**) pour l'interprétation immédiate du danger.
Alerte Sismique	Magnitude, Niveau d'activité	Intégration d'une barre de progression colorimétrique (gradient vert à rouge) pour situer l'intensité du séisme.

2.3 Analyse //////2.3.1 Diagramme de classes

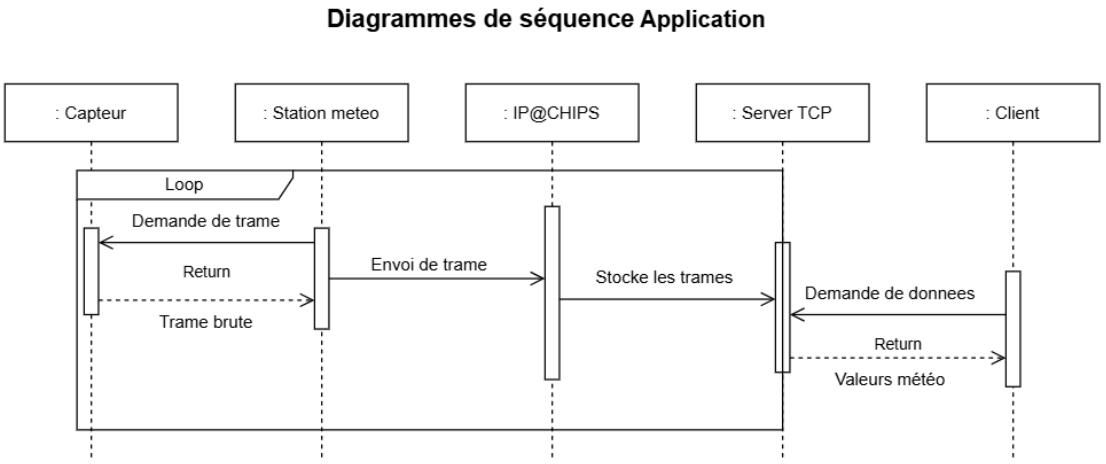
2.3.2 Diagrammes d’états

2.3.3 Diagrammes d’activités

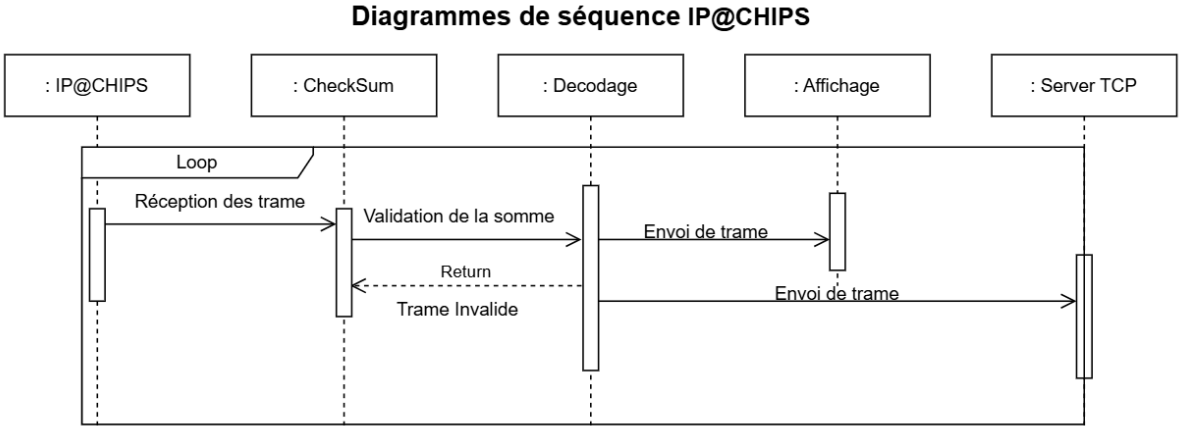


2.3.4 Diagrammes de séquence

2.3.4.1 Diagrammes de séquence Application



2.3.4.2 Diagrammes de séquence IP@CHIPS 16 Bits



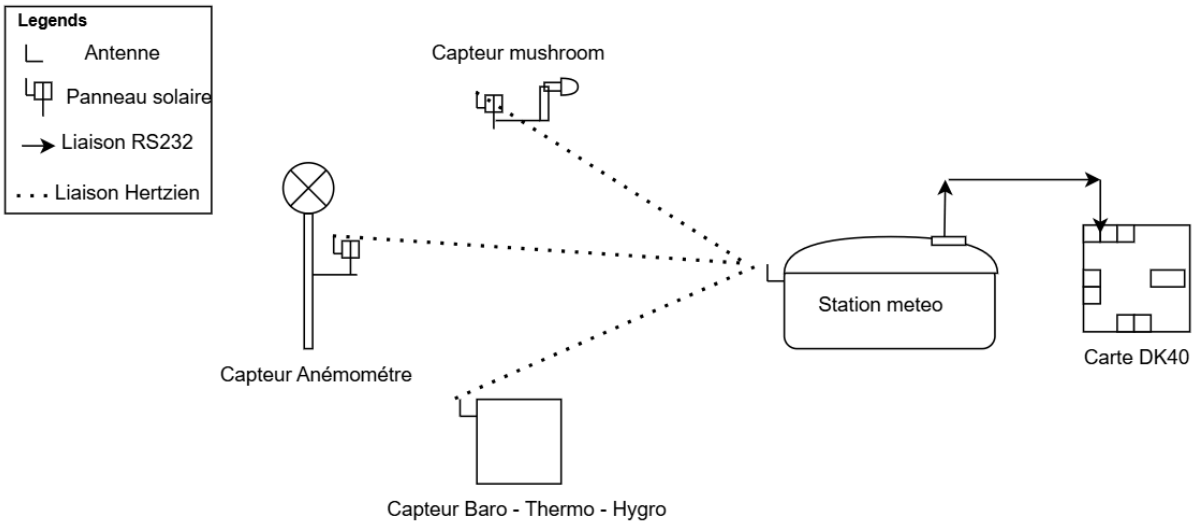
2.3.5 Arborescence de fichiers

```
BC5_WIN7/  Répertoire d'installation
└── AppDos/
    ├── bc5/
    │   ├── BIN
    │   ├── DOC                Chaine de compilation Borland C
    │   ├── HELP
    │   ├── INCLUDE
    │   └── LIB
    ├── Beck/
    │   ├── @CHIP-RTOS documentation Documentation au format html
    │   └── clib Bibliothéques IPV@CHIPS
```

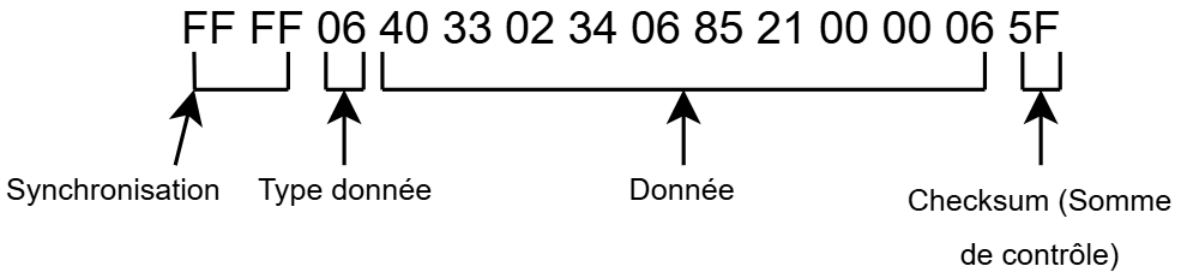
```
└── projets/ Répertoire de projet : Système de fichier MSDOS
└── meteo/ Répertoire d'application Meteo
    ├── decla.c Fichier de déclaration / Prototypes
    ├── fonction.c Ficher de fonction de l'application
    ├── meteo.c Fichier principale(main)de l'application
    ├── compile.bat Fichier de compilation DOS
    ├── METEO.EXE Exécutable de l'application meteo
    └── METEO.OBJ Objet de l'application meteo
```

2.4 Conception

2.4.1 Contexte



Capteur / Équipement	Valeur Mesurer	Rôle
Sonde THB (Thermo-Hygro-Baro)	Température (°C), Humidité (%) et Pression (hPa)	Mesure l'environnement intérieur.
Anémomètre (Capteur Vent)	Vitesse (km/h) et Angle (°)	Capte la force et la direction du vent en extérieur.
Sonde Mushroom (Temp & humidité ext,)	Température extérieure (°C) et Humidité (%)	Fournir une mesure thermique.
Horloge Radio-pilotée	Date et Heure (Synchronisation)	Assure l'horodatage précis des mesures reçues.
Carte DK40	Trames brutes RS232	Reçoit les données série, gère l'automate RTOS et prépare les paquets pour l'IHM.



Le principe de décodage des données (Exemple : Trame THB)

La station météo transmet ses mesures sous forme de BCD (Décimal Codé Binaire). Cela signifie qu'un octet (8 bits) ne contient pas un nombre standard, mais deux chiffres distincts (4 bits par chiffre, appelés "nibbles").

Prenons la partie Données de notre trame : **... 33 02 34 ...**

1. Décodage de la Température (33 02)

La température est codée sur 2 octets et doit être stockée en point fixe (multipliée par 10) pour le RTOS.

Extraction des Dizaines (Octet 02) :

- On isole le chiffre de poids faible avec un masque logique : **0x02 & 0x0F = 2.**
- **Résultat :** 2 dizaines.

Extraction des Unités et Décimales (Octet 33) :

- **Unités :** On décale les 4 bits de poids fort vers la droite **(0x33 >> 4 = 3).**
- **Décimales :** On masque pour garder les 4 bits de poids faible **(0x33 & 0x0F = 3).**

Reconstruction mathématique (Point Fixe) :

- Temp = **(2 * 100) + (3 * 10) + 3 = 233**
- Le RTOS stocke l'entier 233, qui correspondra à 23.3 °C lors de l'affichage.

2. Décodage de l'Humidité (34)

L'humidité est codée sur 1 seul octet.

- **Extraction des Dizaines :** Décalage de 4 bits vers la droite **(0x34 >> 4 = 3).**
- **Extraction des Unités :** Masque logique **(0x34 & 0x0F = 4).**
- **Reconstruction :** (3 * 10) + 4 = 34.
    - Le RTOS stocke directement l'entier 34 (soit 34 %).

3. La Pression Atmosphérique

La logique est la même : extraction des "nibbles" BCD, puis application d'un offset (valeur de base, souvent autour de 856 hPa pour ces capteurs) auquel on additionne la valeur décodée pour retomber sur une pression réelle (ex: 989 hPa).

2.4.2 Extraits de code

2.4.2.1 Extraits de code Application IP@CHIPS 16 Bits

Extraits de code Initialisation de la communication série

```
fossil_setbaud(FOSSIL_EXT, 9600, FOSSIL_PARITY_NO, 8, 1);
fossil_set_flowcontrol(FOSSIL_EXT, FOSSIL_FLOWCTRL_RTSCTS);

printf("Attente de trames meteo\n");

status = fossil_status_request(FOSSIL_EXT);

c = fossil_getbyte(FOSSIL_EXT);
```

Initialisation de la communication série

Cette étape établit les règles de dialogue entre la station météo et la carte IP@CHIPS. La configuration assure une synchronisation parfaite pour que les signaux électriques soient traduits correctement en données numériques sans perte ni corruption.

- **fossil_setbaud** : Configure la vitesse de transmission (9600) ainsi que la structure de la trame (parité, bits de données et de stop).
- **fossil_set_flowcontrol** : Active ou désactive la gestion des signaux matériels RTS/CTS pour réguler le débit entre les deux appareils.
- **fossil_status_request** : Interroge le registre d'état du port COM pour vérifier si un octet a été reçu et attend dans le tampon de réception.
- **fossil_getbyte** : Récupère l'octet le plus ancien présent dans le tampon de réception pour permettre son traitement par la machine d'état.

Extraits de code Machine d'État

```
if (etat == 0) { // Detecter le premier F
            if (octet == 0xFF) etat = 1;
        }
        else if (etat == 1) { // Detecter le Deuxieme FF
            if (octet == 0xFF) etat = 2;
            else etat = 0;
        }
        else if (etat == 2) {
            trame[0] = octet;
            i = 1;
```

Traitement par Machine d'État

Le programme utilise des conditions à états pour isoler les trames valides au milieu du flux de données série. Cette méthode garantit que le décodage ne commence que lorsqu'une séquence spécifique est détectée, évitant ainsi de traiter des données décalées ou incomplètes.

- **État 0 (Synchronisation) :** Le système scrute chaque octet entrant à la recherche du premier marqueur d'en-tête (0xFF).
- **État 1 (Confirmation) :** Si un deuxième 0xFF suit immédiatement le premier, l'en-tête est validé. Sinon, le système retourne à l'état 0 pour éviter une fausse détection.
- **État 2 (Identification)** : Une fois l'en-tête FF FF trouvé, l'octet suivant est enregistré comme étant l'identifiant du capteur (Vent, Température, etc.). Cela permet de définir la longueur de la trame à recevoir.

Extraits de code Identification et Dimensionnement de la Trame

```
if (octet == 0x00)      longueur_attendue = 9;
else if (octet == 0x03) longueur_attendue = 7;
else if (octet == 0x06) longueur_attendue = 12;
else if (octet == 0x0E) longueur_attendue = 3;
else if (octet == 0x0F) longueur_attendue = 7;
else if (octet == 0x01) longueur_attendue = 15;
```

Identification et Dimensionnement de la Trame

L'octet qui suit immédiatement l'en-tête est l'identifiant du capteur. Comme chaque capteur (vent, pluie, température) envoie une quantité d'informations différente, cette étape définit la longueur de la trame à capturer avant de lancer le décodage.

- **0x00 (Anémomètre) :** Trame de 9 octets (vitesse et direction du vent).
- **0x03 (Capteur Extérieur) :** Trame de 7 octets (température et humidité extérieures).
- **0x06 (Capteur Intérieur) :** Trame de 12 octets (pression, température et humidité intérieures).
- **0x0F (Horloge) :** Trame de 7 octets (date et heure de la station).
- **0x01 (Pluviomètre) :** Trame de 15 octets (cumul des précipitations).

Extraits de code d'affichage de la trame de donner en fonction du type

```
else if (etat == 3) {
                trame[i] = octet;
                i++;
```

```
                if (i >= longueur_attendue) {

if (trame[0] == 0x00)      printf("\nTRAME VENT : ");
else if (trame[0] == 0x03) printf("\nTRAME MUSHROO : ");
else if (trame[0] == 0x06) printf("\nTRAME THB : ");
else if (trame[0] == 0x01) printf("\nTRAME PLUIE: ");
else if (trame[0] == 0x0E) printf("\nTRAME MINUTE : ");
else if (trame[0] == 0x0F) printf("\nTRAME CLOCK : ");

                    for(j = 0; j < longueur_attendue; j++) {
                        printf("%02X ", trame[j]);
                    }
                etat = 0;
```

Capture et Finalisation de la Trame

Cette étape assure l'assemblage de l'intégralité des données transmises par le capteur. Le programme accumule les octets dans un tableau jusqu'à ce que la taille prévue soit atteinte, garantissant ainsi l'intégrité du message avant tout traitement.

- **Remplissage :** Chaque octet est stocké séquentiellement dans le tableau trame à l'aide d'un index (i).
- **Identification :** Une fois la trame complète, le programme confirme le type de capteur (Vent, Pluie, THB, etc.) et affiche les données brutes en hexadécimal à des fins de vérification.
- **Réinitialisation du cycle :** L'automate repasse immédiatement à l'État 0. Cette remise à zéro permet de se mettre en attente de l'en-tête FF FF de la trame suivante, assurant une réception en boucle.

2.4.2.3 Extraits de code Application de décodage de trame

Extraits de code fonction Check somme

```
int check_somme(unsigned char *trame, int longueur)
{
    int i;
    unsigned int sum = 0;

    sum += 0xFF;
    sum += 0xFF;

    for(i = 0; i < longueur - 1; i++)
        sum += trame[i];

    return ((unsigned char)(sum & 0xFF) == trame[longueur - 1]);
```

Validation par Somme globale de la trame

Le check somme vérifie l'intégrité de la trame en calculant la somme totale, incluant les octets de synchronisation initiaux. Elle garantit que l'intégralité du message, de l'en-tête jusqu'à la fin des données, a été transmise sans erreur.

- **Réintégration de l'en-tête :** Initialisation de la somme avec les deux marqueurs 0xFF (non présents dans le tableau de données).
- **Masquage 8 bits :** Le résultat final est tronqué pour ne conserver que l'octet de poids faible (somme & 0xFF).
- **Comparaison :** Si le résultat calculé est strictement identique au dernier octet de la trame, la donnée est validée et prête à être décodée.

Extraits de code fonction décodage de la trame VENT

```
const char* DIRECTIONS[] = {"N", "NE", "E", "SE", "S", "SO", "O", "NO" };
void decode_vent(unsigned char *trame) {

    int dizaines = ((trame[2] >> 4) * 10) + (trame[2] & 0x0F);
    int centaines = (trame[3] & 0x0F) * 100;

    int angle = centaines + dizaines;

    int dir_index = ((angle + 22) / 45) % 8;
    int v_unite   = (trame[4] & 0x0F);
    int v_dizaine = (trame[4] >> 4) & 0x0F;
    int v_dixieme = (trame[3] >> 4) & 0x0F;

    float vitesse_ms = (v_dizaine * 10.0) + v_unite + (v_dixieme *0.1);
    float vitesse_kmh = vitesse_ms * 3.6;

    printf("\n--- VENT ---");
    printf("\nAngle     : %d degres", angle);
    printf("\nDirection : %s", DIRECTIONS[dir_index]);
    printf("\nVitesse   : %.1f km/h\n", vitesse_kmh);
}
```

Décodage de l'Anémomètre (Vent)

Cette fonction extrait la vitesse et la direction du vent en utilisant le format BCD (Binary Coded Decimal), où chaque groupe de 4 bits (nibble) représente un chiffre de 0 à 9. Elle convertit ensuite ces mesures brutes en unités standards (km/h et degrés).

- **Calcul de l'angle :** Les dizaines, unités et centaines sont extraites des octets trame[2] et trame[3]. L'assemblage donne un angle compris entre 0 et 359.

- **Détermination de la direction :** L'angle est converti en index (de 0 à 7) pour pointer vers le tableau DIRECTIONS. Le calcul (angle + 22) / 45 permet d'arrondir la direction vers la rose des vents la plus proche (N, NE, E, etc.).
- **Décodage de la vitesse (m/s) :** La vitesse est récupérée en format décimal (dizaines, unités et dixièmes) depuis les octets trame[3] et trame[4].Conversion en km/h : La valeur en mètres par seconde est multipliée par 3.6 pour obtenir la vitesse en kilomètres par heure, puis affichée avec une précision d'un chiffre après la virgule.

Extraits de code fonction décodage de la trame THERMO HYGRO BARO

```
void decode_thb(unsigned char *trame) {
    float temperature = (trame[3] * 10.0) + (trame[2] >> 4) + (trame[2] & 0x0F) * 0.1;
    int humidite = ((trame[4] >> 4) * 10) + (trame[4] & 0x0F);
    int pression = trame[6] + 856;

    printf("\n--- THB ---\n");
    printf("Temperature : %.1f C\n", temperature);
    printf("Humidite    : %d %%\n", humidite);
    printf("Pression    : %d hPa\n", pression);
}
```

Décodage du Capteur Intérieur (THB)

Decode thb convertit les données brutes du capteur interne de la station. Elle utilise un mélange de décodage BCD pour le confort thermique et un calcul d'offset pour la pression atmosphérique.

- **Température (C) :** Elle est reconstituée en extrayant les dizaines (trame[3]), les unités (poids fort de trame[2]) et les dixièmes (poids faible de trame[2]).
- **Humidité (%) :** Calculée en format BCD standard sur l'octet trame[4]. Les 4 bits de gauche donnent la dizaine et les 4 bits de droite les unités.
- **Pression (hPa)** : La station utilise une valeur de base (offset) de 856. On y ajoute la valeur de l'octet trame[6] pour obtenir la pression réelle en hectopascals.

Extraits de code fonction décodage de la trame Mushroo

```
void decode_mushroo(unsigned char *trame) {
    float temp = (trame[3] * 10.0) + ((trame[2] >> 4) * 1.0) + (trame[2] & 0x0F) * 0.1;
    int hum = ((trame[4] >> 4) * 10) + (trame[4] & 0x0F);

    printf("\n--- MUSHROOM ---\n");
    printf("Temp Ext    : %.1f C\n", temp);
    printf("Humidite    : %d %%\n", hum);
}
```

Décodage du Capteur Extérieur (Mushroom)

Decode Mushroo traite les données climatiques provenant de l'unité extérieure. Elle extrait les valeurs de température et d'humidité en isolant les chiffres via des opérations sur les bits (décalages et masquages).

- **Température Extérieure (°C) :** Elle est recomposée à partir de trois éléments : l'octet trame[3] pour les dizaines, les 4 bits de poids fort de trame[2] pour les unités, et les 4 bits de poids faible pour les dixièmes.
- **Humidité (%) :** Comme pour les autres capteurs, l'octet trame[4] est séparé en deux pour obtenir les dizaines et les unités de la valeur d'humidité.

Extraits de code fonction décodage de la trame Date

```
void decode_date(unsigned char *trame_date) {
   int min   = ((trame_date[1] >> 4) * 10) + (trame_date[1] & 0x0F);
   int heure = ((trame_date[2] >> 4) * 10) + (trame_date[2] & 0x0F);
   int jour  = ((trame_date[3] >> 4) * 10) + (trame_date[3] & 0x0F);
   int mois  = ((trame_date[4] >> 4) * 10) + (trame_date[4] & 0x0F);
   int annee = 2000 + ((trame_date[5] >> 4) * 10) + (trame_date[5] & 0x0F);

   printf("\n--- HORLOGE ---");
   printf("\nHeure : %02d:%02d", heure, min);
   printf("\nDate  : %02d/%02d/%d\n", jour, mois, annee);
}
```

Décodage de l'Horloge

Decode date convertit les données temporelles de la station. Contrairement aux capteurs météo, cette trame utilise un décodage BCD (Binary Coded Decimal) sur chaque octet pour représenter les minutes, les heures et le calendrier complet.

- **Heure et Minute :** Chaque unité (heure ou minute) est extraite des octets trame_date[2] et trame_date[1]. Le calcul sépare les dizaines (décalage de 4 bits vers la droite) des unités (masquage 0x0F).
- **Calendrier (Jour/Mois)** : La fonction applique la même logique de séparation des bits pour reconstituer le jour et le mois à partir des octets correspondants.
- **Année :** L'année est calculée sur une base (offset) de 2000. On décode les deux derniers chiffres en BCD et on les ajoute à la base pour obtenir l'année complète.
- **Formatage :** L'affichage utilise %02d pour garantir que les valeurs s'affichent toujours avec deux chiffres.

2.4.2.4 Extraits de code Application multi-thread RTOS.

Optimisation RTOS : Passage du Float au enier INT Dans l'environnement RTOS de la carte IP@CHIPS, l'utilisation du type float est limitée car elle alourdit considérablement la charge processeur. Pour maintenir la précision des mesures météo sans utiliser de nombres à virgule, j'ai utilisé une technique de calcul en int.

Le principe de la "Valeur multipliée par 10"

Au lieu de stocker 22.5 °C, je stocke l'entier 225.

- **Stockage :** La valeur réelle est multipliée par 10 lors du décodage.
- **Calcul :** Les opérations (comme la conversion m/s vers km/h) sont réalisées sur des entiers avec des facteurs multiplicateurs ajustés pour conserver la précision.
- **Affichage :** Lors de l'affichage, on utilise les opérateurs division (/) et modulo (%) pour reconstruire la virgule visuellement :
    - **valeur / 10** pour la partie entière.
    - **valeur % 10** pour la décimale.

Extraits de code création du sémaphore

```
int g_mutex;
result = RTX_Create_Sem(&g_mutex, "SEM_MET", 1);
if(result != 0) {
  printf("Erreur creation semaphore %d\n", result);
  return 0;
}
```

Gestion de la ressource partagée (Mutex)

Ce bloc initialise le "verrou" indispensable pour sécuriser l'accès aux données globales dans un environnement multitâche (RTOS).

- **RTX_Create_Sem :** Crée un sémaphore nommé "SEM_MET" pour faciliter l'identification du verrou dans le noyau du système.
- **Exclusion mutuelle :** En réglant la valeur initiale à 1, le sémaphore agit comme un Mutex. Il garantit qu'une seule tâche à la fois (Décodage ou Afficher) peut manipuler la structure g_meteo, évitant ainsi toute corruption de données.
- **Sécurité système :** Le test sur result vérifie que l'OS a bien alloué la ressource. En cas d'échec, le programme s'arrête immédiatement pour éviter un comportement imprévisible.

Extraits de code création de la structure DATA

```
typedef struct {
  // CAPTEUR
  int temp;
  int temp_ext;
  int vent_kmh;
  int angle;
  int pression;
  int humidite;
  int humidite_ext;
```

```
  char dir_nom[5];

  // HORLOGE
  int heure;
  int minute;
  int jour;
  int mois;
  int annee;
  int flag_date;
} Data;
```

Structure de données globale (Data)

Cette structure permet de partager les informations entre les tâches. Elle centralise toutes les mesures traitées par le décodage pour les rendre disponibles au reste du système (serveur TCP, affichage).

- **Données Capteurs :** Stockage des mesures météorologiques (température, vent, pression, humidité). L'utilisation exclusive du type int confirme l'implémentation du point fixe pour optimiser les performances du RTOS. Direction du Vent : Le champ dir_nom[5] stocke la traduction textuelle de l'angle (ex: "NE", "SO") pour un affichage immédiat sans recalcul.
- **Horloge:** Regroupement des variables temporelles (heure, date) pour assurer la traçabilité des mesures.
- **Flag de statut :** Le champ flag_date permet aux autres tâches de savoir si une trame de temps valide a été reçue et décodée.

Extraits de code création de la Mise à jour de la structure temporelle

```
  RTX_Wait_Sem(g_mutex, 0);
    g_meteo.minute = min;
    g_meteo.heure  = heure;
    g_meteo.jour   = jour;
    g_meteo.mois   = mois;
    g_meteo.annee  = annee;
    g_meteo.flag_date = 1;  // date reçue
  RTX_Release_Sem(g_mutex);
```

Ce segment de code intervient juste après le décodage BCD de l'horloge. Il permet d'enregistrer les informations de date et d'heure dans la structure globale tout en garantissant la cohérence des données pour les autres tâches.

Fonctionnement :

- **Verrouillage (RTX_Wait_Sem) :** Avant de modifier la structure g_meteo, la tâche demande l'accès exclusif au sémaphore. Si une autre tâche (comme l'affichage) est déjà en train de lire, le décodeur attend que la ressource se libère.
- **Enregistrement des données :** Les variables (minute, heure, jour, etc.) sont copiées dans les champs correspondants de la structure partagée.
- **Activation du flag_date :** Le passage de ce drapeau à 1 est crucial ; il signale aux autres modules du programme (IHM, Serveur TCP) que la station a réussi sa synchronisation temporelle et que les données de date sont désormais valides.
- **Libération (RTX_Release_Sem) :** Le verrou est relâché immédiatement après l'écriture pour permettre au reste du système d'accéder aux nouvelles informations.

Extraits de code creation de la task_decode et task_affiche

```
#define STACK_SIZE 1024
unsigned int stack_decode[STACK_SIZE];
TaskDefBlock def_decode = {
    task_decode,
    {'T','D','E','C'},
    stack_decode,
    STACK_SIZE,
    0,
    1,
    0,0,0,0,0
};
result = RTX_Create_Task(&id_decode, &def_decode);
if(result != 0) {
    printf("Erreur creation task_decode %d\n", result);
    RTX_Delete_Sem(g_mutex);
    return 0;
}
unsigned int stack_affiche[STACK_SIZE];
TaskDefBlock def_affiche = {
    task_affiche,
    {'T','A','F','F'},
```

```
    stack_affiche,
    STACK_SIZE,
    0,
    1,
    0,0,0,0,0
};
result = RTX_Create_Task(&id_affiche, &def_affiche);
if(result != 0) {
    printf("Erreur creation task_affiche %d\n", result);
    RTX_Delete_Sem(g_mutex);
    return 0;
```

Création et Paramétrage des Tâche de Décodage et Affichage

Ces blocs définissent comment le système doit exécuter la fonction de traitement en arrière-plan.

- **Allocation de Pile (STACK_SIZE) :** Réserve un espace mémoire dédié (1024 mots) pour les tâche. Cet espace stocke les variables locales et le contexte d'exécution au fonction task_decode, task_affiche.
- **TaskDefBlock (Configuration) :**
    - **Identifiant :** La tâche sont nommées 'T','D','E','C et 'T','A','F','F pour être identifiables dans les outils de debug de l'OS.
    - **Priorité :** Définit l'importance de la tâche face aux autres processus.
- **RTX_Create_Task :** Enregistre la tâche auprès du noyau. Une fois créée, la fonction
- **Gestion d'Erreur :** Si la tâche ne peut pas démarrer (manque de mémoire, par exemple), le programme libère proprement le sémaphore précédemment créé (RTX_Delete_Sem) avant de s'arrêter pour éviter de laisser des ressources "fantômes"

Extraits de code de la task_decode

```
void huge task_decode(void) {
    int a, etat = 0, i = 0, len = 0, timeout = 0;
    unsigned char c, trame[30];
    while(1) {
        if(fossil_status_request(FOSSIL_EXT) & 0x01) {
            a = fossil_getbyte(FOSSIL_EXT);
            if(a != -1) {
```

```
            c = (unsigned char)a;
            switch(etat) {
                case 0: if(c==0xFF) etat=1; break; // Synchro 1
                case 1: if(c==0xFF) etat=2; else etat=0; break; // Synchro 2
                case 2: trame[0]=c; i=1; len=0; etat=3; break; // ID Capteur
                case 3:
                    trame[i++] = c;
                    // Attribution de la longueur selon l'ID (0x06, 0x00, 0x03, 0x0F...)
                    if(trame[0]==0x06) len=12;
                    else if ... // [Identification des autres types de capteurs]
                    if(len>0 && i>=len) {
                        if(check_somme(trame, i)) {
                            // Routage vers le décodeur approprié
                            if(trame[0]==0x06) decode_thb(trame);
                            else if ... // [Appel des fonctions decode_vent, mushroom, etc.]
                        }
                        etat=0; i=0; // Reset pour la prochaine trame
                    }
                    break;
            }
            timeout = 0;
        }
    } else {
        if(++timeout > 500) { etat = 0; i = 0; timeout = 0; } // Sécurité timeout
    }
    RTX_Sleep_Time(1); // Relâchement du CPU pour l'ordonnanceur
  }
}
```

Task_decode s'exécute en boucle infinie en arrière-plan. Son rôle est de surveiller le port série, d'isoler les messages valides et de déclencher les bons décodeurs.

Points clés de l'implémentation :

- **Gestion du Driver Fossil :** La fonction utilise fossil_status_request pour vérifier la présence de données sans bloquer le système. Si un octet est disponible, il est récupéré par fossil_getbyte.
- **Les États (Switch/Case) :**

    - **États 0 et 1 :** Recherche de la double synchronisation 0xFF 0xFF.
    - **État 2 :** Capture de l'identifiant du capteur.
    - **État 3 :** Accumulation des données jusqu'à atteindre la longueur len spécifique à chaque capteur (THB, Vent, Mushroom ou Date).
- **Sécurité par Timeout :** Si la station s'arrête de transmettre au milieu d'une trame (parasite ou déconnexion), un compteur timeout réinitialise l'automate à l'état 0 après 500 cycles d'inactivité. Cela évite que le programme ne reste "bloqué" dans l'attente d'un octet qui n'arrivera jamais.
- **Validation :** Une fois la trame complète, le check_somme est vérifié. Si la trame est saine, la fonction appelle le décodeur spécifique (decode_thb, decode_vent, etc.).
- **RTOS (RTX_Sleep_Time) :** L'appel à RTX_Sleep_Time(1) à chaque itération est crucial. Il permet de rendre la main à l'ordonnanceur pour que les autres tâches (comme le serveur l'affichage) puissent s'exécuter sans ralentissement.

Extraits de code de la task_affiche

```
void huge task_affiche(void)
{
   Data local;

   while(1)
   {
      RTX_Wait_Sem(g_mutex, 0);
         local = g_meteo;
      RTX_Release_Sem(g_mutex);

      if(local.flag_date){
      printf("\n============= STATION METEO | %02d/%02d/%d %02d:%02d
============",local.jour, local.mois, local.annee,local.heure, local.minute);
      } else {
      printf("\n============= STATION METEO | ---/--/---- --:-- =============");
      }

      printf("\n================ CAPTEUR BARO-TERMO-HYGRO  ================");
      printf("\n   Temp : %d.%d C", local.temp / 10, local.temp % 10);
      printf("\n   Hum  : %d %%", local.humidite);
      printf("\n   Pres : %d hPa", local.pression);

      printf("\n===================== CAPTEUR MUSHROOM
=====================");
```

```
    printf("\n   Temp Ext : %d.%d C", local.temp_ext / 10, local.temp_ext % 10);
    printf("\n   Hum Ext  : %d %%", local.humidite_ext);

    printf("\n======================= CAPTEUR VENT ======================= ");
    printf("\n   Vitesse : %d.%d km/h", local.vent_kmh / 10, local.vent_kmh % 10);
    printf("\n   Angle : %d", local.angle);
    printf("\n   Direction : %s", local.dir_nom);

    RTX_Sleep_Time(1000);
}
```

Task_affiche Cette tâche gère l'interface utilisateur locale (console). Elle illustre l'utilisation du Mutex pour lire les données de manière sécurisée et la technique du point fixe pour l'affichage des décimales.

Points clés de l'implémentation :

- **Lecture Sécurisée :**
    - La tâche utilise RTX_Wait_Sem et RTX_Release_Sem pour copier l'intégralité de la structure globale g_meteo dans une variable locale local.
    - **Pourquoi ?** Cela permet de libérer le Mutex le plus vite possible pour ne pas bloquer la tâche de décodage, tout en travaillant sur une "photo" stable des données.

Reconstruction des Flottants (Point Fixe) :

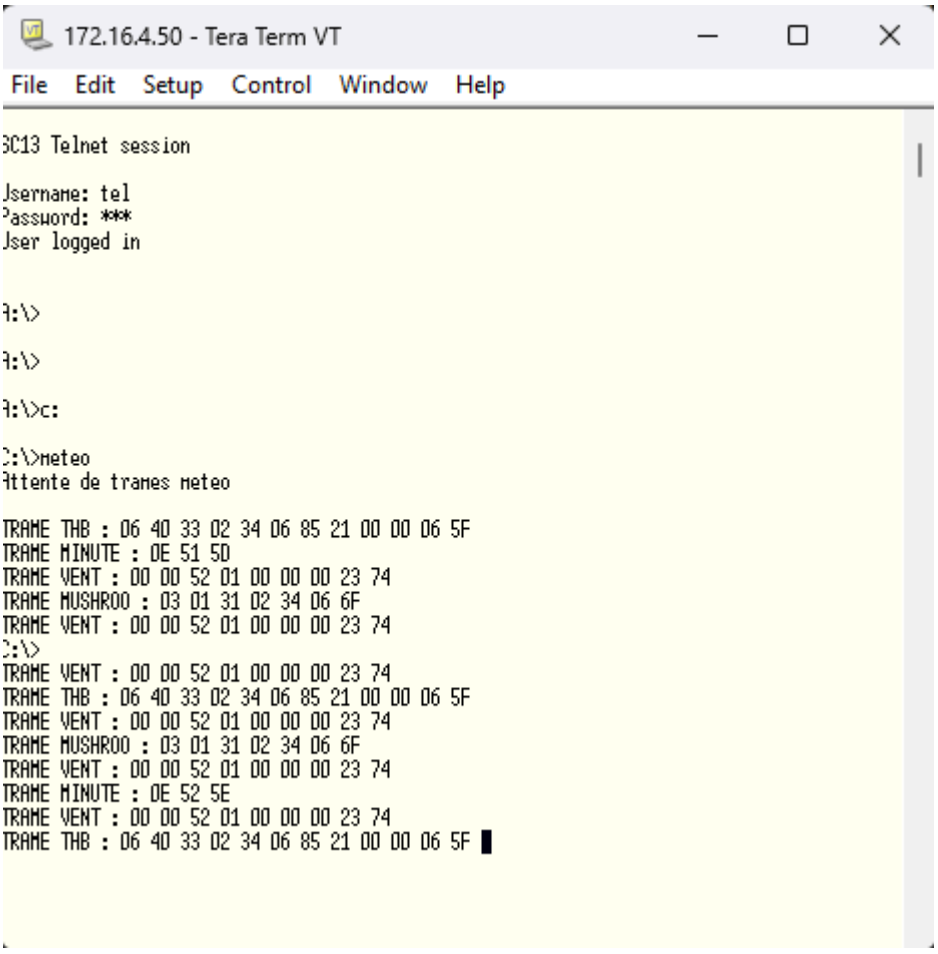
- Comme les valeurs sont stockées sous forme d'entiers multipliés par 10 (ex: 225 pour 22.5 °C), l'affichage utilise :
    - **local.temp / 10** pour la partie entière.
    - **local.temp % 10** pour récupérer le chiffre après la virgule.

- **Logique d'Affichage Conditionnelle :**
    - Le programme vérifie le flag_date. Si aucune trame de temps n'a encore été reçue, il affiche des tirets (--/--) au lieu de valeurs erronées, garantissant la fiabilité des informations affichées.
    - **Pourquoi ?** Car les trame d'horloge dite clock sont transmise toute les heure et l'équipement utiliser ne nous permette pas de modifier la fréquence d'émission des trame comme désirer.
- **Temporisation (RTX_Sleep_Time) :**

- La tâche s'endort pendant 1000 ms (1 seconde) entre chaque rafraîchissement. Cela évite de saturer le processeur et la console inutilement, les capteurs météo n'évoluant pas à la milliseconde.

2.5 Tests unitaires

2.5.1 Resultat d’execution Application IP@CHIPS 16 Bits

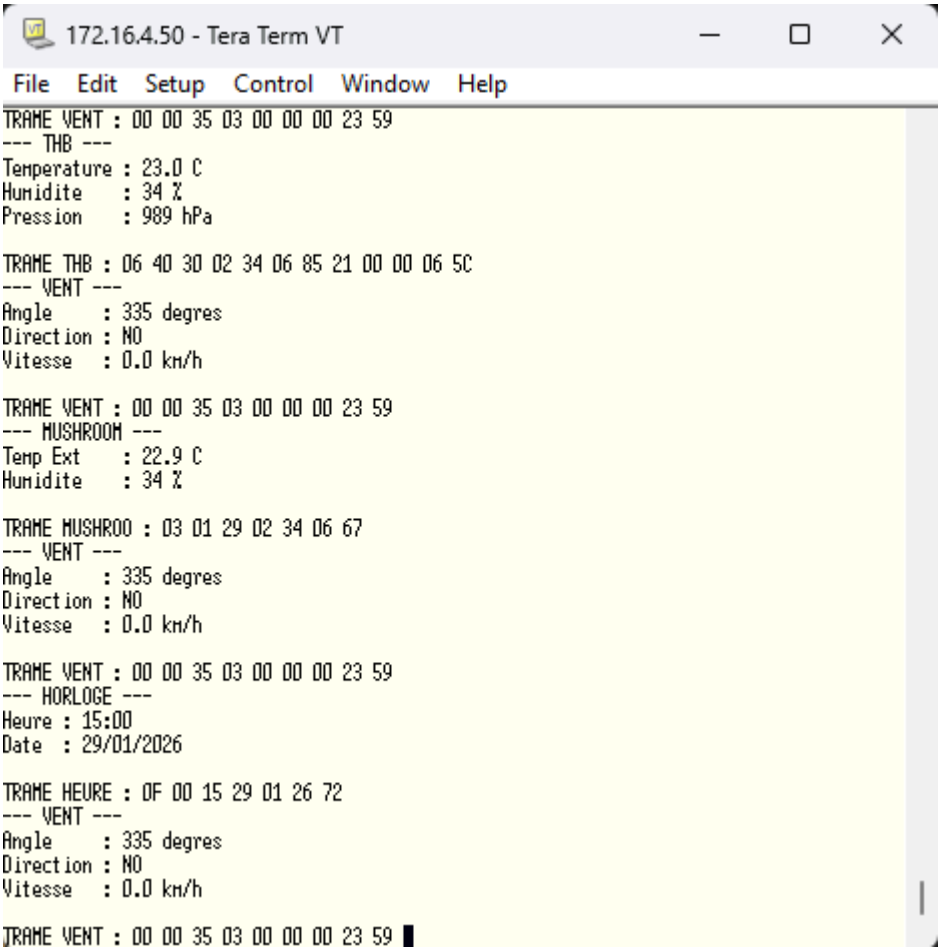


Validation par les Tests

La capture d'écran du terminal Tera Term montre le bon fonctionnement :

- **Identification précise :** Le programme reconnaît instantanément le type de trame reçu (VENT, THB, MUSHROO, MINUTE).
- **Contrôle du format :** Chaque trame est extraite avec sa longueur exacte (ex: 12 octets pour le THB, 9 pour le vent), prouvant que le découpage des données brutes est bon.
- **Affichage des données brutes :** L'affichage en hexadécimal permet de confirmer que les octets reçus correspondent bien aux spécifications du constructeur avant leur décodage en clair.

2.5.2 Resultat d’execution Application de décodage et affichage des données de la station meteo

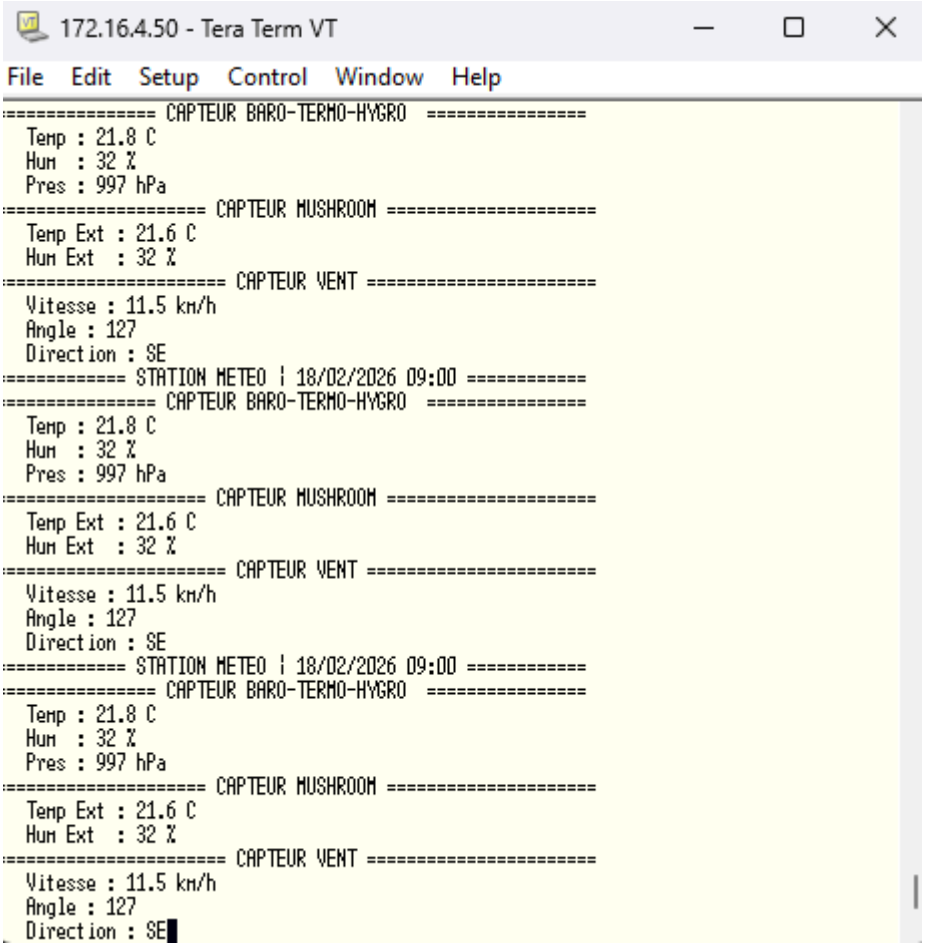


Validation des résultats

La capture d'écran du terminal Tera Term montre le bon fonctionnement du decodage :

- **Identification et Longueur :** L'automate de réception découpe les octets en respectant la longueur spécifique à chaque capteur.
- **Décodage en temps réel :** On observe les données hexadécimales en valeurs lisibles (ex: Temperature : 23.0 C, Heure : 15:00). Cela valide les fonctions de calcul et la gestion de l'horloge.

2.5.3 Resultat d’execution système multithread décodage et affichage météo RTOS

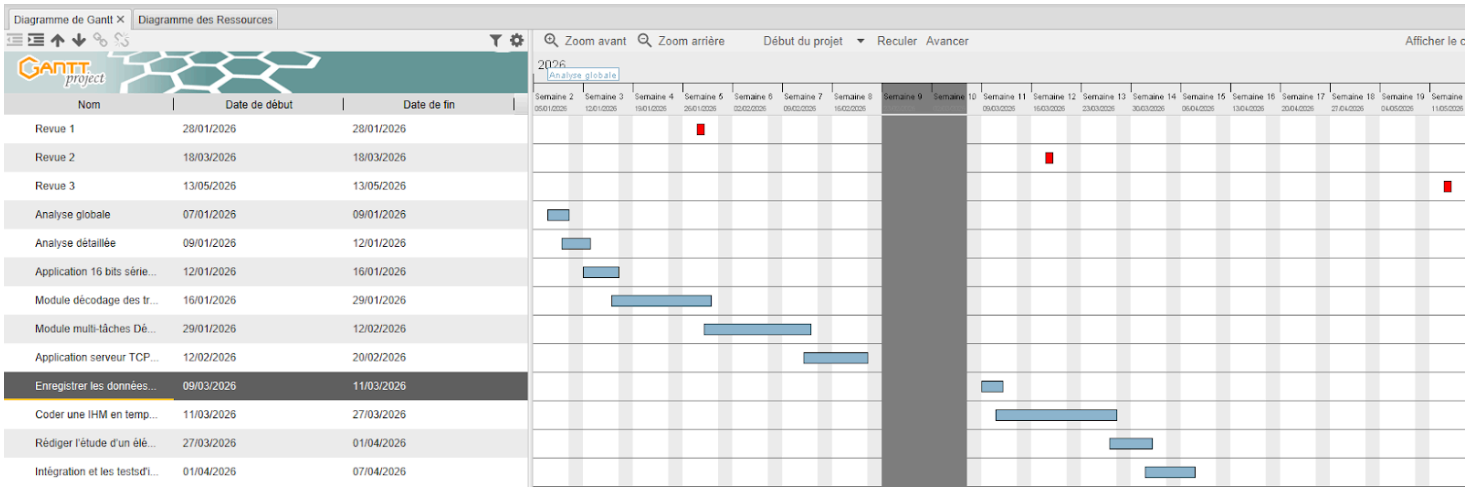


2.6 Diagramme de Gantt

2.6.1 Planning prévisionnel

GANT SURVEILLANCE ENERGETIQUE ET ENVIRONNEMENTALE	2026																				
Semaine	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**	**S7**	**S8**	**S9**	**S10**	**S11**	**S12**	**S13**	**S14**	**S15**	**S16**	**S17**	**S18**	**S19**	**S20**	**S21**
Revue 1																					
Revue 2																					
Revue 3																					
Remise du projet																					
Effectuer l'analyse globale																					
Effectuer l'analyse globale détaillée candidat 1																					
Effectuer l'analyse globale détaillée candidat 2																					
Effectuer l'analyse globale détaillée candidat 3																					
Effectuer l'analyse globale détaillée candidat 4																					
Coder une application 16 bits série																					
Coder le décodage des trames météo																					
Coder multithread décodage météo																					
Coder serveur météo multithread																					
Coder enregistrement des données météo dans SGDB																					
Coder IHM supervison météo, UV, particules, sismique																					

2.6.2 Planning réaliser



2.6.2 Carnet de bord hebdomadaire des tâches réalisées

2.7 Intégration

2.7.1 Tests d'intégration

2.8 Partie physique

2.9 Conclusion

➢ Faire un bilan des aspects fonctionnels des tâches que vous aviez à réaliser